CSBR: A Compositional Semantics-Based Service Bundle Recommendation Approach for Mashup Development

Qi Gu, Jian Cao, *Senior Member, IEEE*, and Yancen Liu, *Student Member, IEEE*

Abstract—An increasing number of services are being offered which leads to difficulties in choosing appropriate services during mashup development. Currently, several service recommendation techniques have been developed for mashup creation, however, they are largely limited to suggesting services which have similar functionalities. The fundamental problem with these techniques is that they do not consider the large semantic gap between mashup descriptions and service descriptions. In this article, we propose a compositional semantics-based service bundle recommendation model (CSBR) to tackle this problem. CSBR is based on a semantic service package repository, which is constructed by mining the existing mashups. Specifically, the reusable service packages, which consist of multiple collaborative services, are annotated with composite semantics rather than their original semantics. Based on the semantic service package repository, CSBR can recommend a bundle of services that cover the functional requirements of the mashup as completely as possible. Extensive experiments are conducted on a real-world dataset and the results show CSBR achieves significant performance improvements in both precision and recall metrics over the state-of-the-art methods.

Index Terms—Mashup creation, compositional semantics, service bundle recommendation

1 Introduction

Recent years have witnessed continuous development in service computing, causing the number of Web services to grow rapidly on the World Wide Web. Despite the large number of various services available, we still encounter a situation where a single service cannot satisfy the users' complex requirements. For example, most weather services take a city's postal code as input, however, the postal code is often not known beforehand. To use weather services, we have to first obtain the postal code by invoking another service. Therefore, service composition (i.e., reusing existing services and combining them into new services) is a significant technology in service-oriented computing and mashup has become a popular approach in service composition implementations [1], [2].

Mashup is a developer-centric technique which can combine services together to create web applications with more comprehensive functions. In addition, successful mashups can be published and reused. Mashup development is one of the end-user development (EUD) models which helps non-IT professionals repurpose the existing services to develop comprehensive applications for themselves [3]. Several mashup development environments were developed by major Internet companies, such as Microsoft, IBM and Google, to empower end-users to create mashups. When using a mashup development environment, the creators can quickly compose services via several "drag-and-drop" actions through a modern easy-to-use web-based UI. For example, *NaturalMash* is a live mashup tool built for intuitive compositions with a What You See Is What You Get (WYSIWYG) interface [4], which enables non-professional users to develop useful and non-trivial mashups. Consequently, the development cycle and cost associated with a mashup are drastically reduced.

Despite the aforementioned convenience, it is still challenging to select appropriate services for end-users during mashup development. This is due to the large number of services currently available on the Web (e.g., programmableweb.com[1] had published 23,990 services by March 2021). As is often the case, users type the words that describe the functionality of the mashup they desire to develop, nevertheless, they often become lost in the numerous services. Not surprisingly, in recent years, many recommendation approaches have been proposed to provide help [5], [6], [7], [8], [9], [10], [11], [12]. Generally, these approaches are based on the explorations of relationships between a mashup and services. Consequently, in the recommended service set, there is often more than one service with similar functions. In order to rank similar services, [5], [6], [8] identify the top-k most probable services by integrating the popularity of services. Moreover, candidate services can also be ranked by category [13], [14].

Even though these methods can help users discover useful services, they are not sufficient for mashup creation. On

- *Qi Gu is with the School of Information Science and Technology, Nantong University, Nantong, Jiangsu 226001, China, and also with the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai 200240, China. E-mail: guqi@sjtu.edu.cn.*
- *Jian Cao and Yancen Liu are with the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai 200240, China. E-mail: {cao-jian, LiuYancen}@sjtu.edu.cn.*

Manuscript received 31 May 2020; revised 9 Apr. 2021; accepted 21 May 2021. Date of publication 1 June 2021; date of current version 9 Dec. 2022. (Corresponding author: Jian Cao.)
Digital Object Identifier no. 10.1109/TSC.2021.3085491

1. https://www.programmableweb.com

1939-1374 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:59:02 UTC from IEEE Xplore. Restrictions apply.

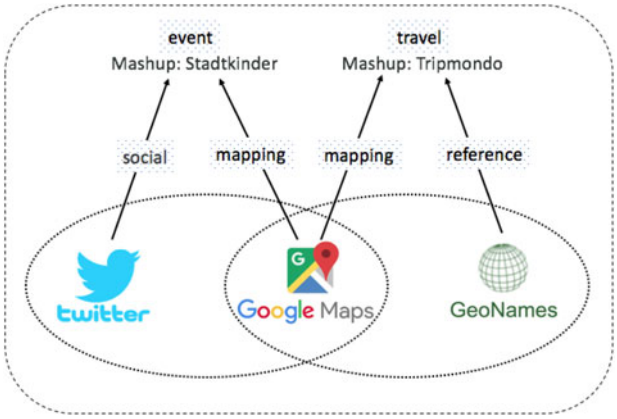


Fig. 1. An example of the semantics of mashups and their component services.

one hand, it is still a difficult task for a user to select the best one from similar services. On the other hand, users need multiple services that can cooperate with each other to meet their requirements. Few of existing methods consider the relationships between recommended services. As a matter of fact, the co-occurrence of services in an existing mashup indicates the inner relationships between services. Therefore, a straightforward way to recommend a group of services is to identify services that have co-occurrence relationships, which can be discovered by association rule mining. [15] takes advantage of historical composition patterns to support service recommendation. Since these services have been put together in many mashups, they are very likely to collaborate with one another again in the future. However, this approach fails to recommend services that never collaborate before. Another approach is to recommend a service package in which each service matches different parts of the description of the mashup so that these services complement each other[16]. Moreover, it can consider the frequent co-occurrence relationships at the same time to further improve the recommendation performance[17], [18]. Unfortunately, these approaches can not model the compositional relationships between services sufficiently.

To recommend a bundle of complementary services that try to cover all the functional requirements of a mashup to the user is non-trivial and poses a set of challenges.

The first challenge derives from the semantic gap between the functional descriptions of services and the functional description of a mashup. In particular, the semantics of a service package is not simply the addition of the semantics of its constituent services. More often than not, a package of collaborative services depicts a semantics that is very different from their original semantics. As shown in Fig. 1, the function "mapping" provided by *Google Maps API* and the function "social" offered by *Twitter* generate the function "event" of their mashup *Stadtkinder*, the function "mapping" provided by *Google Maps API* and the function "reference" supported by *GeoNames* lead to the function "travel" of their mashup *Tripmondo*. Therefore, a package of collaborative services can generate a compositional semantics, which can describe their integrated function. How to learn the compositional semantics of a package of collaborative services is still unexplored.

The second challenge comes from the computation complexity. Compared with single service recommendation, service bundle recommendation is essentially a combinatorial optimization problem. It is difficult to find a bundle of collaborative services that can meet the requirements of the mashup well from the large space consisting of an increasing number of services. Roughly speaking, if there are s candidate services, and the maximum length of a bundle is set to b, there will be $\Sigma_{i=1}^{b} C_s^i$ possible compositions. Therefore, we need a method to efficiently generate and choose candidate bundles for developers, rather than applying ranking methods to all possible bundles directly.

To address the challenges, we present the compositional semantics-based service bundle recommendation framework (CSBR) for mashup development. The main contributions of this paper are as follows:

–We introduce a method to learn the compositional semantics of a service package. To the best of our knowledge, this is the first effort to leverage the compositional semantics in the service recommendation for mashup development. Specifically, we discover potential reusable service packages and learn their compositional semantics from existing mashups and save them into a repository. In particular, the compositional semantics of service packages are learned from elementary discourse unit (EDU) segmentations, topic modeling and semantic clustering.

–Based on the semantic service package repository, we propose an optimization model for recommending a bundle of services which tries to cover the semantics of a given specification for a mashup to be developed as completely as possible. Additionally, we learn the service packages from existing service composition, which guaranteed the recommended services can be integrated.

–We perform extensive experiments on a real-world dataset and compare it with state-of-the-art methods. The experiment results show significant improvements in both precision and recall metrics.

The rest of this paper is organized as follows. Section 2 summarizes the related work. We describe the proposed model in Section 3. Section 4 presents the construction process of the semantic service package repository. Section 5 introduces the optimization model for service bundle recommendation. Section 6 reports the experiment results on a real-world dataset. Finally, we conclude the paper in Section 7.

2 Related Work

Service recommendation for mashup development has attracted the attention of many researchers and we divide the related work into two categories, i.e., service recommendation and service bundle recommendation.

2.1 Service Recommendation

Traditional information retrieval is a major method for selecting appropriate services based on the matching of description. Approaches based on editing distance, vector space model and TF-IDF were used to build a service search engine in the early days. However, this service matching procedure usually suffers from poor performance. Therefore, semantic approaches are proposed to improve performance

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:59:02 UTC from IEEE Xplore. Restrictions apply.

[19], for example, to make use of the semantic input-output information in service discovery [20], [21], to add semantic annotations to service descriptions [22] and to improve the description based on an ontology framework [23]. Although these approaches add semantics to the descriptions in order to achieve better service matching performance, this approach is hard to apply to the large-scale services.

As natural language processing technology has developed quickly, more advanced technologies have also been used to extract the semantic relationships between mashups and services. [5] proposed an approach to explore the semantic relationships between mashups and services based on the topic model. In [7], the description is refactored using the Author-Topic Model to eliminate the gaps between mashup developers and service developers due to development contexts and development vocabularies. As we have discussed, an information retrieval-based approach only considers the superficial semantics. Our approach also relies on the topic model to represent the semantic information of both mashups and services. However, our approach tries to learn the compositional semantics of service packages in order to cover both superficial and compositional semantics at the same time.

More complex models such as stacked denoising autoencoders (SADE) are used to extract features for recommendation in [24]. [25] adopts the LSTM model to learn and predict user's preferences and service features for recommendation. [26] designs a recommendation algorithm based on a representation learning method of a knowledge graph. [27] and [28] both use the attention network in a factorization machine to predict the ratings between mashups and services. Although these approaches try to learn more latent relationships between mashups and services through the deep learning model, they ignore the compositional semantics of composed services. In addition, these approaches still recommend a list of services instead of a bundle of services.

2.2 Service Bundle Recommendation

Instead of providing the information of a list of similar services, service bundle recommendation method recommends a bundle of compatible services to satisfy the functional requirements of the mashup as completely as possible. Our approach also falls into this category.

An intuitive approach takes advantage of frequent service compositions as recommendations. This is quite straightforward since a frequent service composition means that these services are put together in many mashups, consequently, they are very likely to collaborate with one another again in the future. These compositions could be discovered by applying many traditional data mining approaches such as association rule mining. For example, in [15], a recommendation approach for mashup development via frequent pair mining is proposed. In [16], the techniques to mine reusable composition patterns and recommend services via pattern matching and retrieval are adopted. Occasionally, this approach can satisfy the requirement of deeper composition relationships. However, recommendation precision can not be guaranteed since the function requirements of a mashup are not considered. At the same time, this approach suffers from two other limitations. One is that this approach needs some services to have already been chosen. The second is if two services have never previously cooperated with each other before, they cannot be recommended.

In addition, some researchers have tried combining the frequent service composition based approach with information retrieval based approach. For example, a multi-level relational network is proposed to capture and model the comprehensive relationships between topics, tags and services [18]. In addition to the descriptions, tags and their relationships, mashups and services also have some additional information[29]. For example, the popularity of services and the provider information of services can also help improve recommendation performance. Therefore, classical recommendation algorithms such as collaborative filtering and matrix factorization are combined with other techniques to make full use of this information. [6], [30] combines the LDA model and collaborative filtering to recommend a list of services for users. In [9], the hierarchical Dirichlet process and probabilistic matrix factorization are integrated using Bayes theorem and ranked services are recommended to users based on popularity. The relational topic model and factorization machine are utilized to recommend services for mashup development in [31]. In [32], the topics of mashup is modeled at content level and network level.

Moreover, some technologies try to identify the social relations between mashups and services. For example, [8] proposes a manifold ranking algorithm which incorporates the relationships between mashups and services. In [33], [34], [35], the inner connections between services are discovered for recommendations. In [17], we proposed an approach for recommending a set of services whose relationship are consistent with the relationships among sentence blocks of the textual description of a desired mashup. In [13], independent service clustering is provided as the basis of service recommendation. Unfortunately, in all these methods, the collective semantics of the collaborative services, i.e., a service package have not been learned and used, which is the focus of the method proposed in this paper.

3 A Compositional Semantics-Based Service Bundle Recommendation Approach

3.1 Problem Definition

When deciding to design a mashup, the developer has to decide which functionalities should be offered. However, it is difficult to express the functionalities in structured or fomalized form. As is often the case, descriptive texts are more widely used to describe the functionalities of mashup. Meanwhile, various mashup information sharing platforms provide rich descriptions to describe their functionalities. Hence, function goals can be extracted from textual descriptions using natural language processing techniques.

Let us consider a scenario in which a developer describes its functions in natural language, such as "*show you information you might be looking for, as you type. It gets web search results to inform you about related stuff on the net, related news happening around the world, Flickr photos to help you visualize, and Amazon product recommendations*." Since the number of services available is huge, it is a challenging work to find appropriate services. Service recommendation approaches mainly focused on recommending a list of services based on

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:59:02 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Notations Used in This Paper

SYMBOL	DESCRIPTION
M	a set of mashups, $M = \{m_1, m_2, \ldots, m_{\lvert M\rvert}\}$
S	a set of services, $S = \{s_1, s_2, \ldots, s_{\lvert S\rvert}\}$
W	a set of words w appearing in the descriptions of a mashup m in M (W_m) or a service s in S (W_s)
S_m	the set of services s in mashup m
P_m	the set of service packages p in mashup m
E_m	the set of EDUs e generated from the description of mashup m
B_m	a bundle of service packages p of mashup m
$\mathcal{C}_p$	the compositional semantics of service package p
$\mathcal{E}_p$	the explicit semantics of service package p
U_e	indicates that if EDU e is covered by compositional semantics
g	the maximal size allowed for a service package
p	a service package
k	number of clusters
s_p	a service which is popular
s_{up}	a service which is unpopular
N_b	number of recommended bundles
N	number of recommended services
T	number of topics z

the textual description. Unfortunately, these approaches suffer from the problem that many services with similar functions are recommended. Developers have to select appropriate services among them. A more efficient way is to recommend a bundle of cooperative services as a whole, i.e., service bundle recommendation.

As a service bundle recommendation system, it analyzes the textual description, searches appropriate component services from a service repository and then recommends a bundle of compatible services to the developer, such as: *Yahoo Term Extraction* that extracts keywords from user search records, *Yahoo Search* that finds relevant information based on these keywords, *Flickr* which presents pictures of this information to the user, and *Amazon Product Advertising* recommends products to users according to the relevant information.

Therefore, the service bundle recommendation problem is defined as follows. Each mashup $m \in M$ itself will consist of one or more services. The goal of service bundle recommendation is, given the textual description of a mashup m, a bundle of services $B_m \subseteq S$ of size $|B_m|$ is recommended to the user, where B_m includes services s from service repository S. In this paper, we focus on how to recommend a service bundle that has a high probability of being accepted by developers. At the same time, this service bundle can satisfy the functional requirements of this mashup as completely as possible.

Table 1 summarizes the notations used in this paper.

3.2 The Overview of the Framework

The service bundle recommendation framework can be divided into two subsystems, as illustrated in Fig. 2, i.e., the semantic service package acquisition system (SSPA system) and the service bundle recommendation system (SBR system).

In the SSPA system, the textual descriptions of existing mashups as well as their services are analyzed in the first phase. The descriptions are split into element discourse units (EDUs) based on a logistic regression classifier. The topic model is then leveraged to train on EDUs so that each EDU is shaped as a topic vector. In the second phase, we divide each mashup into a combination of service packages based on different package sizes (i.e., different size service packages). EDUs are roughly assigned to the corresponding service packages. Then, clustering and filtering operations are utilized to extract the compositional semantics for each service package. By doing so, a semantic service package repository can be constructed.

In the SBR system, a bundle of services can be recommended to satisfy the mashup's functional requirements. Given the textual descriptions of the new mashup's functionalities, its EDUs and the corresponding topic vectors are generated. Then a bundle of service packages is recommended with each package matching an EDU. Specifically, a greedy algorithm is proposed to find the service bundles that best match the EDUs.

4 Semantic Service Package Repository Construction

Service bundle recommendation is based on a semantic service package repository. Therefore, in this section, the key techniques for constructing this repository are introduced.

4.1 Extracting Semantics With the Topic Model

Semantics is a bridge to connect mashup descriptions and services so that we should learn and represent the semantics of mashups and services. Since existing mashups and services are often supplied with textual descriptions, we can extract semantics from these textual descriptions.

However, mashup descriptions usually include a mixture of its component functions. In Rhetorical Structure Theory (RST) [36], a discourse tree (DT) is posited to represent the discourse of natural text. In a DT, leaves are the contiguous atomic text spans, called Elementary Discourse Units, a rhetorical relation (e.g., elaboration) to link the adjacent EDUs. We split the textual description of mashup into a sequence of EDUs $(e_1, e_2, \ldots)$, where EDU e_i is the unit that serves as a building block for the whole textual description. For example, consider the following textual description of mashup *Tipmondo*:

Tripmondo is a travel information browser for trip destinations across the globe. It features videos, photos, guides and attractions, weather information and events for nearly all countries and many cities world wide. Tripmondo is based on CMSmadesimple.

This textual description can be partitioned into 3 EDUs and the detail is exemplified by the discourse tree of mashup *Tripmondo* shown in Fig. 3.

To segment the textual description of mashups into EDUs, we need to determine whether a word in a sentence is the EDU boundary. Thus, a logistic regression classifier with parameter Θ is applied to decide the EDU boundary [37] of a mashup. If w is the EDU boundary, then $y = 1$, else $y = 0$.

$$P(y|w, \Theta) = Bernoulli(y|Sigmoid(\Theta^T \mathbf{x})), \tag{1}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:59:02 UTC from IEEE Xplore. Restrictions apply.

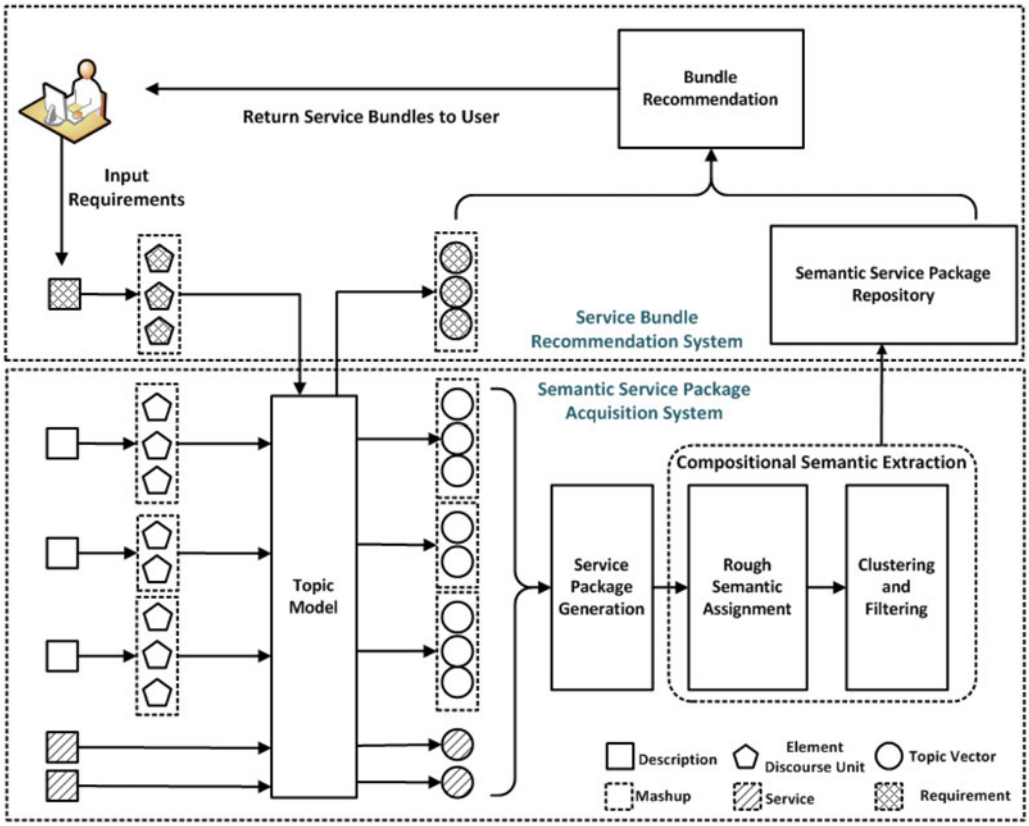


Fig. 2. The overview of the framework for service bundle recommendation.

where x denotes the lexico-syntactic feature vector of word w in the mashup description extracted from the lexicalized syntactic tree, and if w is the EDU boundary, then $y = 1$, else $y = 0$. The loss function with l_2 regularization is given by

$$J_{log}(\theta) = -\sum_{i}^{n} y^{(i)} \log(\hat{y}^{(i)}) - (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) + \alpha\theta^T\theta, \tag{2}$$

where $y^{(i)}$ denotes the lable for word $w^{(i)}$ and the model parameter Θ is learned from the limited-memory BFGS fitting algorithm based on the Standard RST-DT corpus.[2] 5-fold cross validation is used to avoid overfitting. Based on Eq. (1), the textual description of a mashup is segmented into a sequence of EDUs.

In order to derive the latent useful topics of the descriptions of mashups and services, we make use of a generative probabilistic model – latent Dirichlet allocation (LDA) [38]. LDA can be utilized to model the hidden topics of documents, which is a statistic model to discover the topics of a series of documents [39]. As a result, we represent the EDUs and the description of services as a mixture of topics, where each topic is a probability distribution over words. We determine the number of topics based on the theorem proposed in [40].

The major shortcoming of topic model is the stablility of topics (also known as order effects [41], i.e., different topics are generated if the order of training data is shuffled). [41] proposed a tool (LDADE[3]) to tune LDA parameters using differential evolution to incease the clustering stability of standard LDA. As [41] only use the result of one LDA run, [42] proposed a method that relies on replicated LDA runs. In this paper, we utilized the method that relies on replicated LDA runs proposed in [42]. Once the number of topics T is determined, we run LDA on the EDUs of all the mashups and related web services.

4.2 Reusable Service Package Discovery and Mashup Segmentation

A *service package* is a subset of services in a mashup that can provide a function and each mashup can be segmented into a combination of service packages. The segmentation is not unique for a mashup. We try to find service packages that can be reused with high probability in the future. It is straightforward to see that a service package that appears more frequently in the whole mashup repository is more likely to be reused. We count the appearance frequency $freq(p_j)$ of each service package p_j.

How to segment a mashup into service packages in the best way can be modeled as an optimization problem. Therefore, the mashup segmentation problem and its solution are described as follows:

Given a mashup m and its associated service set S_m, we can obtain the possible service package set P_m (each package includes at least one of the services in S_m and we suppose P_m includes total n packages). Suppose mashup m can be segmented into different service package combinations

2. https://catalog.ldc.upenn.edu/LDC2002T07 3. https://github.com/amritbhanu/LDADE-package

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:59:02 UTC from IEEE Xplore. Restrictions apply.

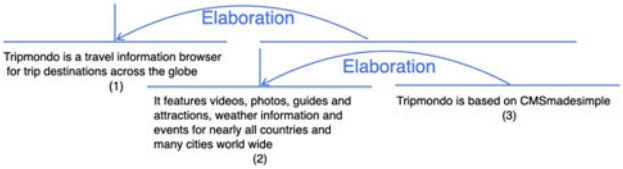


Fig. 3. The discourse tree of mashup tripmondo.

B_m. The problem of finding the optimal combination which covers all the members of S_m by elements from P_m without overlapping is denoted as,

$$\begin{aligned} &\underset{y}{\text{maximize}}\, y = \frac{\sum_{j=1}^{n} weight_j x_j}{\sum_{j=1}^{n} x_j} \\ &\text{subject to} \sum_{j=1}^{n} a_{ij} x_j = h \\ &x_j \in \{0,1\} \quad (j = 1, 2, \ldots, n), \end{aligned} \tag{3}$$

where $weight_j = freq(p_j)size(p_j)$ is the weight corresponding to $p_j \in P_m$, a_{ij} is 1 if the ith item of the service set S_m is present in service package p_j and 0 otherwise, x_j is a binary decision variable that is set to 1 if package p_j is included in the solution and to 0 otherwise, h is an $|S_m|$ column of ones, and $n = 2^{|S_m|} - 1$. The constraint ensures that a service in the service set can only be present in one service package selected for inclusion in the admissible group.

Note that $freq(p_j)$ denotes the frequency of service package p_j appearing in the whole service repository. Since the frequency of a small-size service combination must be greater than that of a large-size service combination, we add the size of service package ($size(p_j)$) into the numerator to enhance the weight of the large-size service combination, and also add the number of service packages ($\sum_{j=1}^{n} x_j$) in B_m into the denominator as a limitation to minimize the size of B_m.

This is essentially a set partitioning problem (SPP), so we use the Gurobi toolkit [43] to solve it. Consequently, a mashup can be segmented into a combination of service packages. For example, mashup *Tripmondo* with services {*Eventful, Google Maps, YouTube, GeoNames, Twitter, Panoramio, Weather Channel, Wikipedia, Currencies Exchange Rates, Google Maps Data*} can be segmented into {Google maps, Geonames}, {wikipedia, Eventful}, {Twitter, youtube}, {weather channel, google maps data, panoramio}, and {Currencies Exchange Rates}.

4.3 Compositional Semantics Annotation for Service Packages

Based on the aforementioned two techniques, semantics are extracted from the mashup descriptions and each mashup is also segmented into service packages. Now we discuss how to assign the extracted semantics to each service package.

A simple way to learn the semantics of a service package is to aggregate the semantics of the corresponding services directly. However, simply collecting the semantics of the services cannot reflect their collective semantics. This is why we need to learn the semantics from EDUs of the mashup. Unfortunately, a mashup consists of multiple service packages and the corresponding relationships between EDUs and service packages are not straightforward. First, for an EDU, if its semantics is closely correlated with the semantics of the services within a service package, this service package is likely to inherit the semantics of this EDU.

Superficial semantic similar relationship represents the relationship between an EDU and a service package when the semantics of the EDU is strongly associated with the semantics of services in a service package directly.

In our method, if the similarity between an EDU and a service package is greater than the threshold value γ, the relation between them is defined as a semantically similar relationship. To calculate the similarity degree, we compute the scores between services s in service package p ($p \in B_m$) and EDU e in E_m

$$Sim(p, e) = \max_{s \in p} \frac{\mathbf{v}_s \cdot \mathbf{v}_e}{|\mathbf{v}_s||\mathbf{v}_e|}, \tag{4}$$

where v_s is the topic vector of service s and v_e is the topic vector of EDU e. Dot production is a popular measure of similarity in the information retrieval domain. Then the semantics of the EDU is assigned to the service package when they have a superficial semantically similar relationship. Thus, given a service package p, the semantics inherited from EDU is denoted as

$$\mathcal{E}_p = \bigcup_{e \in E_m} \{e | Sim(p, e) \geq \gamma\}. \tag{5}$$

When there is no superficial semantic similar relationship between an EDU and a service package, their relationship has two possibilities. One possibility is this EDU is not related to the service package at all. Another possibility is the semantics of the EDU can represent the collective semantics of all the services, rather than the semantics of any single service in the package. This collective semantics is called compositional semantics.

Compositional semantics represents the collective semantic meanings generated by the services of a service package. This function cannot be provided by any single service. Instead, this function relies on all the services as a whole. For example, as shown in Fig. 1, "travel" is a compositional semantics of the service package [*Google Maps, GeoName*] despite "travel" not being directly related to the original semantics of these two services. To find which service package can be assigned a compositional semantics, we assign the semantics to all service packages first and then filter out those unrelated ones through a clustering procedure. The clustering and filtering process is shown in Fig. 4.

As shown in Fig. 4, a given service package is attached with a set of semantics which are represented by the circles. We divide these semantics into k clusters according to the distances between different semantics. We try to keep the semantics in a semantic cluster as tightly connected as possible, and make the distances between the semantic clusters as large as possible. We use forward k-means clustering to partition semantics of service packages.

Given the multiple semantics of service package p, where each semantics is a T-dimensional vector, k-means aims to partition these semantics into k sets $\{sc_1, sc_2, \ldots, sc_k\}$ so as to minimize the square error$J(J = \sum_{i=1}^{k} \sum_{\mathbf{x} \in sc_i} ||\mathbf{x} - \mu_i||_2^2)$,

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:59:02 UTC from IEEE Xplore. Restrictions apply.

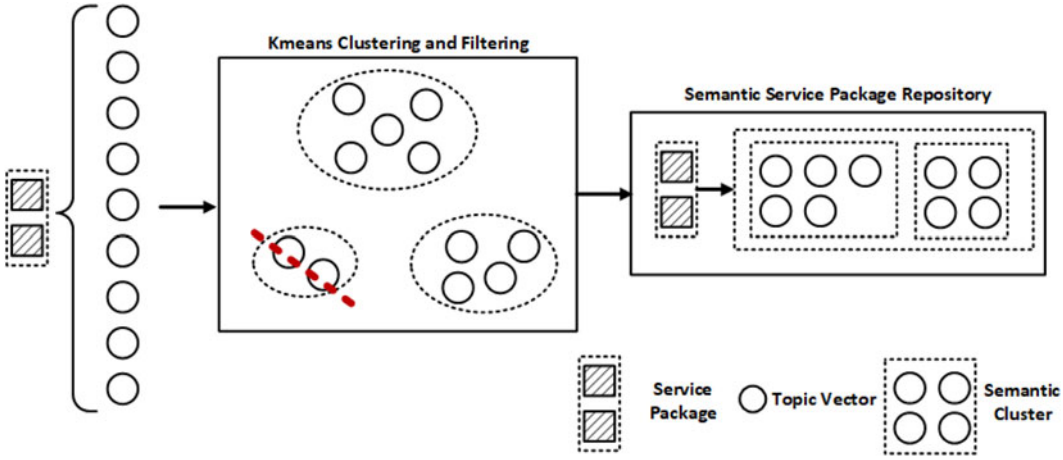


Fig. 4. The clustering and filtering process.

whereμ_i ($\mu_i = \frac{1}{|sc_i|}\sum_{\mathbf{x}\in sc_i}\mathbf{x}$) is the mean vector of semantic cluster sc_i. The cluster number k is decided by the elbow method [44]. Here, we use euclidean distance as a metric. The k-means algorithm proceeds by alternating between two steps: (i) to assign each semantics to the cluster whose mean has the least euclidean distance, and (ii) to calculate the new means to be the centroids in the new clusters. The algorithm converges when the assignments no longer change.

Some semantic clusters are obtained from its constituent services and EDUs who have superficial semantically similar relationships. Regarding the other clusters, if their sizes are relative large, they are more likely to represent the compositional semantics of this package. The reason for this is that similar compositional semantics will often appear in mashups which include this service package. For example, the service package $[GoogleMaps, GeoNames]$ appears in 42 mashups, of which, 19 mashups are related to the semantics "travel" as shown in Table 2.

Therefore, if the size of a cluster reaches a predefined threshold level, it can be regarded as compositional semantics. Taking mashup $[Furkot]$ in Table 2 as an example, the description of mashup $[Furkot]$ is "*Plan your road trip by clicking on the map: Calculate driving time, find attractions, reserve lodging.*", where "*plan your road trip*" is semantically similar to "*find and map the best road rides*" and "*discover new rides*" in mashup $[RideDog]$. After clustering, the semantics can be divided into clusters. Since 19 of the 42 mashups in the service group $[GoogleMaps, GeoNames]$ are travel-related, it can be believed that this service package generates the semantics "travel" and this cluster will not be removed in the filtering step. In contrast, unrelated semantics are often scattered and they only form smaller clusters because they don't appear often in different mashups. Therefore, unrelated semantics can be filtered by removing these small clusters.

In the implementation, we train the EDUs and historical services with the Stanford Topic Modeling Toolbox (TMT).[4] After this, the semantic blocks of each mashup and service are shaped as topic vectors. Using the aforementioned process, all service packages are assigned the compositional semantics. The detail is summarized in Algorithm 1.

Algorithm 1. Semantic Service Package Repository Construction

Input: M: Mashup set
B_m: the combination of service packages in mashup m
Output: $\mathcal{C}$: the constructed semantic service package repository

```
Initialize L    //L stores the semantics of all the service packages
for each m in M do
    for each e of E_m do
        if exists p ∈ P_m and Sim(p, e) ≥ γ then
            assign the semantics of e to l_p
        else
            assign the semantics of e to all the service packages in P_m
        end if
    end for
end for
for each l_p in L do
    if |l_p| ≥ δ then
        for each cluster in Kmeans(k, l_p) do
            if |cluster| ≥ |l_p|/μ then
                assign cluster to c_p
            end if
        end for
    else
        assign l_p to c_p
    end if
end for
return C
```

5 Service Bundle Recommendation

Given the textual description of mashup m to be developed, we separate it into EDUs and generate their topic vectors. Then, the similarity between service packages and the semantics of EDUs is calculated. If the similarity is over a given threshold ϕ, the semantics of EDU is similar to the semantics of the service package, i.e., the semantics of this EDU can be covered by the service package. Our goal is to find bundle B_m where the service package p in B_m can cover

4. https://nlp.stanford.edu/software/tmt/tmt-0.4/

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:59:02 UTC from IEEE Xplore. Restrictions apply.

TABLE 2
Mashups With [GoogleMaps,GeoNames] Package Related to the Semantics "Travel"

INDEX	NAME	DESCRIPTIONS
1	Trip Organizer	A 360 degree view of a location while organizing a vacation or business trip...
2	WeGooo	WeGooo wants to help you live better in the city you're in or the city you want to be in...
3	G4ng	From the site: The G4 is a powerful tool for everybody who wants to keep his/her geo-spatial...
4	VirtualTraveller	Virtualtraveller aims to supply the user relevant information about some location on...
5	Carpool Connect	Free carpool matching service based on home location and work location.
6	PlaceMix	Travel search mashup portal that lets users track all the information about their favorite places...
7	Travel the World!	Social travel guide using Foursquare, GeoNames and Google Maps.
8	Geowalk	A useful worldwide travel guide in German and English. With embedded data from ...
9	Mappington	A social community for travelers. Create travel blogs, post your photos & videos...
10	Furkot - free online road trip planner	Plan your road trip by clicking on the map: calculate driving time, find attractions...
11	OK Travel Australia	okTravel is an Australia travel web-site with the goal of displaying statistical data as culture.
12	TripMixx	STravel search portal like iGoogle. Find flights, hotels, weather, maps, photos, videos...
13	Tripmondo	Tripmondo is a travel information browser for trip destinations across the globe. It features...
14	Geowalk Light	Download and install this map on your own site. Double-click anywhere on the map for ...
15	Cheap Flights To	Cheapflights-to.net is a travel tools for buying airline tickets to destinations, package...
16	Rutamina	Network for routes, travels, bikes, cars, and more. Integrated with social networks such as...
17	Road Trip	gives you all level-1, the US states, and level-2, the US counties, administrative areas...
18	Wolpy	Track your own and your friends travels.
19	RideDog	RideDog aims to help cyclists find and map the best road rides, mountain bike trails and...

as many semantics as possible, and at the same time, the duplication of semantics between service packages is minimized.

It can be modeled as an optimization problem, as illustrated in the following:

$$\begin{aligned} &\underset{y}{\text{maximize}}\, y = \sum_{i=1}^{n} r_i sim_i - \sum_{i \neq j} r_i r_j dup_{ij}, \\ &\text{subject to} \sum_{i=1}^{n} r_i = d, \\ &r_i \in \{0,1\} \quad (i = 1, 2, \ldots, n), \end{aligned} \tag{6}$$

where r_i indicates whether package i is selected in the bundle, n is the number of service packages in the SSPA system, Sim_i denotes the semantic similarity score between the package and the mashup description, and dup_{ij} denotes the semantic overlapping score between service package p_i and service package p_j, which can be computed using the Jaccard coefficient.

$$dup_{ij} = \lambda \frac{|p_i \cup p_j|}{|p_i \cap p_j|}, \tag{7}$$

where λ controls the tradeoff between the similarity and overlapping. The higher the score, the larger the overlap. d denotes the number of recommended service packages in a bundle.

This problem is NP-hard. Fortunately, d is small in most cases, so this problem can be reduced to the mixed integer quadratic programming problem [45]. We can apply a global search algorithm to this problem and use the Gurobi toolkit [43] to solve it. However, there is still a high time complexity. As shown in Fig. 5, the statistics conclude that d is quite small in the mashup development scenario. Consequently, the method can be accomplished using a greedy approximation algorithm. We propose Algorithm 2 which

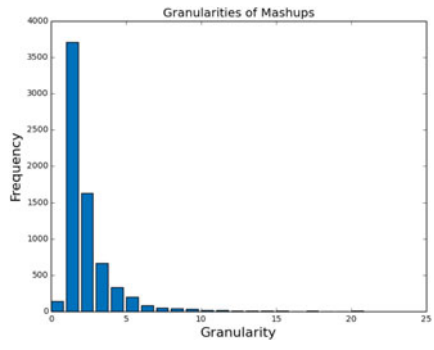


Fig. 5. The frequency of different sizes of mashups.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:59:02 UTC from IEEE Xplore. Restrictions apply.

will find a relative better result among the prefix list during each run.

Algorithm 2. The Greedy Algorithm for Service Bundle Recommendation

Input: m: a new mashup with a user's description
g: the maximal size allowed for service packages
P: the constructed service package set
$\mathcal{C}$: the constructed semantic service package repository
α and β: the parameters
ϕ: the threshold
Output: a recommended service bundle B_m

1: Parse the description of mashup m into EDU set E_m
2: Initialize U $//U_e$ represents if e is covered
3: Initialize $\mathcal{C}^g$ $//\mathcal{C}^g$ represents all the service packages whose size is no more than g
4: **repeat**
5: Initialize maximum heap H //heap H stores $< Sim(p), p >$
6: **for** each p in P **do**
7: **for** e in E_m **do**
8: **if** $U_e = 1$ **then**
9: $Sim(p) = Sim(p) - \beta * Similarity(\mathcal{C}_p, e)$
10: **else**
11: $Sim(p) = Sim(p) + \alpha * Similarity(\mathcal{C}_p, e)$
12: **end if**
13: **end for**
14: add $< Sim(p), p >$ into heap H
15: **end for**
16: $Flag = False$
17: **while** heap H is not empty **do**
18: Extract $< Sim(p), p >$ from heap H
19: **for** each e in E_m **do**
20: **if** $Similarity(\mathcal{C}_p, e) \geq \phi$ and $U_e = 0$ **then**
21: $U_e = 1$
22: $Flag = True$
23: **end if**
24: P.remove(p)
25: **end for**
26: **if** Flag **then**
27: add p into B_m
28: break
29: **end if**
30: **end while**
31: **until** Flag=False
32: **return** B_m

Given the text description for a new mashup m, in Step 1, the description is parsed into an EDU set based on a discourse segmenter and set E_m is obtained. A vector U for each EDU e is generated in E_m, which denotes if e is covered by any service package in P in Step 2. From Steps 4 to 29, bundle B_m is found where the service package p in B_m can cover as many semantic blocks as possible, and at the same time, the semantic overlap between p and m is minimized. Finally, the results are returned in Step 30. The time complexity is $O(|B_m| \times |E_m| \times |\mathcal{C}^g| \times log(|\mathcal{C}^g|))$, where $|B_m|$ is the number of service packages in a bundle and $|E_m|$ is the number of EDUs in mashup m and $|\mathcal{C}^g|$ denotes the number of service packages in the semantic service package set that includes all the service packages whose length is no more than g.

TABLE 3
Statistical Properties of the Dataset

Number of mashups	6,979
Number of services	12,140
Number of unique words	21,485
Number of services appearing in at least one mashup	1,134

6 Experiments

To the best of our knowledge, ProgrammableWeb.com is the largest mashup information sharing community. We crawled 6,979 mashups and 12,140 services (called APIs in ProgrammableWeb.com) from ProgrammableWeb.com. Each mashup and each service contain their descriptions, which describe their functionalities. We evaluate the proposed algorithm based on this dataset.[5] Table 3 summarizes the statistical properties of the crawled dataset.

6.1 Comparative Methods

To verify the effectiveness of the proposed bundle recommendation approach, we compare CSBR with seven state-of-the-art methods described as follows.

–*WVSM*. The weighted vector space model is a word-based approach. In this method, services and mashups are represented by binary vectors v_s and v_m. The distance between a mashup and a service can be calculated by computing the cosine similarity between their vector representations. At the same time, the popularity of the service $popularity_s$ is utilized as a weight. Then WVSM ranks the services according to the product of similarity and weight.

That is

$$p_{wvsm}(m, s) = popularity_s \frac{\mathbf{v}_m \mathbf{v}_s}{||\mathbf{v}_m|| ||\mathbf{v}_s||}. \tag{8}$$

–*WJaccard*. Weighted Jaccard is a word-based approach. In this method, a Jaccard similarity coefficient is defined as the number of common terms shared by the mashup and service divided by the number of unique terms contained by either the mashup or the service. The popularity of the service is also utilized as a weight. Then WJaccard ranks services according to the product of similarity and weight.

That is,

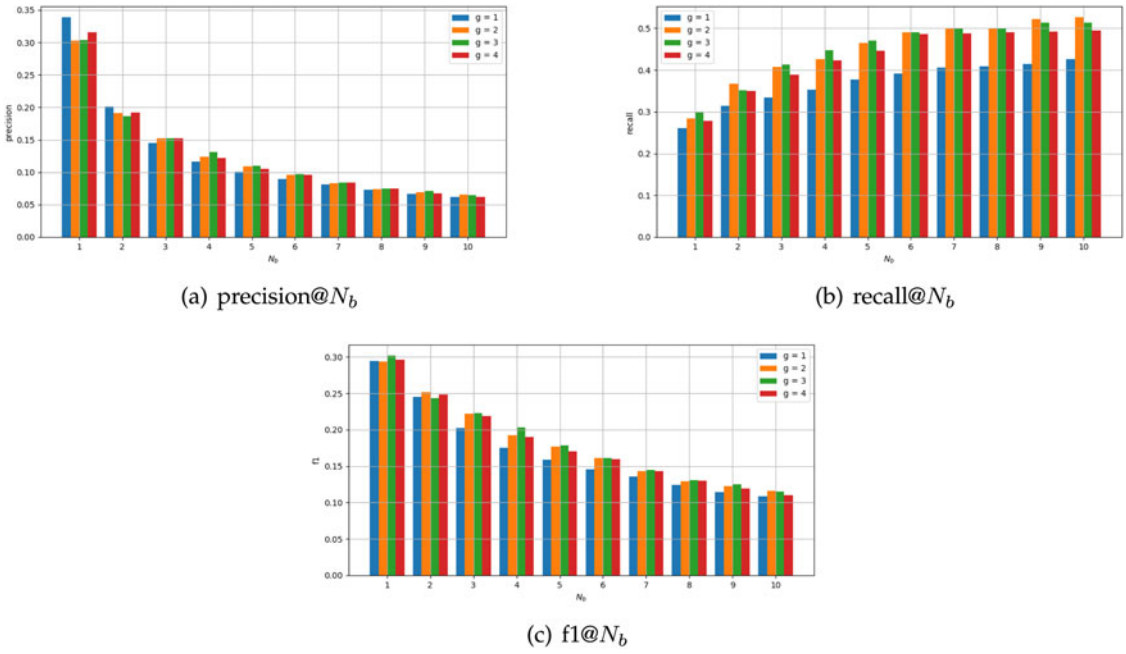
$$p_{WJaccard}(m, s) = popularity_s \frac{|W_m \cap W_s|}{|W_m \cup W_s|}, \tag{9}$$

where W_m and W_s represent the set of words appearing in the description of a mashup m or a service s, respectively.

–*CF*. The CF method applies the collaborative filtering [46] technique for recommendation. Collaborative filtering is one of the traditional recommendation methods, where the items bought by one user may be recommended to a user with a similar purchase history. In this method, mashups play the role of users and services play the role of items. We compare the similarity between the user's requirements and the description of the existing mashups.

5. The code and datasets are available at https://github.com/BaccanoEva/CSBR

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:59:02 UTC from IEEE Xplore. Restrictions apply.



(a) precision@N_b (b) recall@N_b

(c) f1@N_b

Fig. 6. Comparative results of bundle recommendation based on different package sizes.

In this way, the services used in the most similar mashups are recommended.

–*ERTM*. The enhanced relational topic model [5] leverages the relational topic model, which uses the probabilistic topic model – latent Dirichlet allocation to extract the relationships among tags, mashups and services.

–*TopicCF*. TopicCF is designed to combine the topic model and collaborative filtering approach [6]. This method aggregates functionality, usage history and the popularity of services to estimate the recommended relevant services.

–*CDSR*. CDSR [13] first clustered services into categories based on their functionalities and popularities, then map the requirements to relevant categories, and finally combined the services in the most relevant categories to generate a set of candidate services.

–*MRN*. MRN [18] constructed a multi-level relational network to capture deep relationships among services and support service recommendation. They formalized the service package recommendation as a QKP problem and solve it using Branch and Bound algorithm.

6.2 Evaluation Metrics

To evaluate the effectiveness of the proposed approach, some evaluation metrics in traditional service recommendation algorithms are considered. To evaluate the accuracy of $top\text{-}N$ service recommendations for mashup, we choose $precision@N$ and $recall@N$. We also evaluate the accuracy of $top\text{-}N_b$ service bundle recommendation of mashup. The two most common metrics are $precision@N_b$ and $recall@N_b$,

$$precision@N_b = \frac{1}{|gt|}\sum_{m\in gt}\frac{1}{N_b}\sum_{i=1}^{N_b}\frac{|B_m^i \cap gt_m|}{|B_m^i \cup gt_m|}, \tag{10}$$

meanwhile,

$$recall@N_b = \frac{1}{|gt|}\sum_{m\in gt}\frac{1}{N_b}\sum_{i=1}^{N_b}\frac{|B_m^i \cap gt_m|}{|gt_m|}, \tag{11}$$

where B_m^i denotes the ith recommended bundle of the given mashup m, gt_m denotes the ground truth of the given mashup m, and N_b denotes the number of recommended bundles.

Precision measures the ratio of recommendations preferred by users, while recall measures the degree to which the needs of users have been met. Since we take into account the concept of "bundle", our target is to cover as many services as users may need. Consequently, we prefer recall to measure our performance, while precision is used for assistance.

Meanwhile, we compute the evaluation metrics $f1@N_b$, which considers both precision and recall by computing their harmonic mean. The equation of $f1@N_b$ is presented as follows:

$$f1@N_b = \frac{2 * precision@N_b * recall@N_b}{precision@N_b + recall@N_b}. \tag{12}$$

Moreover, we also compute the mean average precision (MAP), which is a widely used evaluation metric in recommender systems. MAP measures the ranking of the recommendation list which also influences recommendation quality. Here, MAP@N can be computed as

$$MAP@N = \frac{1}{|gt|}\sum_{m\in gt}\frac{1}{|S_m|}\sum_{i=1}^{N}\left(\frac{N_i}{i}\right), \tag{13}$$

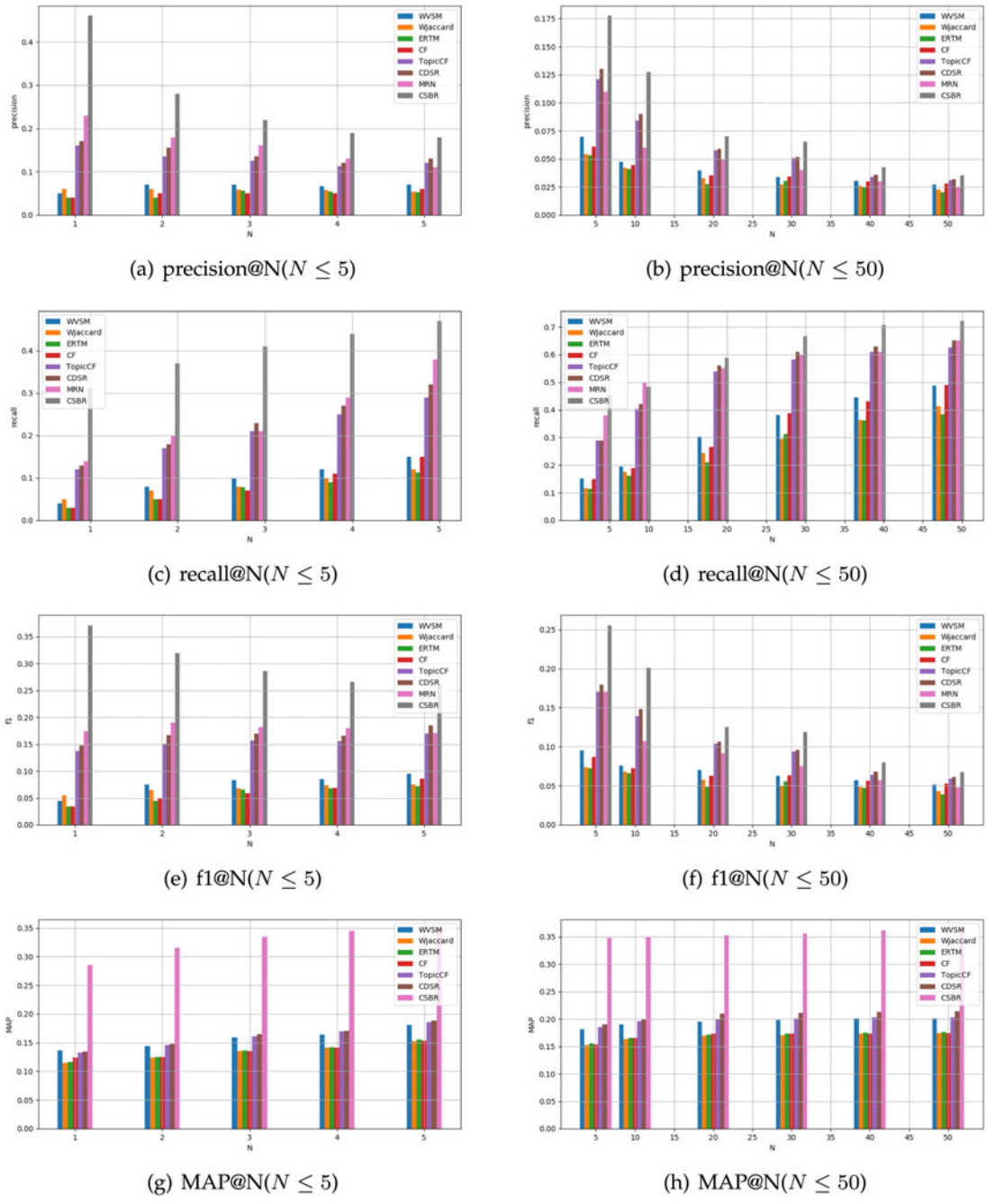
where $|S_m|$ is the number of component services in mashup m, gt is the ground truth ($m \in gt$), $|gt|$ is the number of mashups in gt, and N_i is the number of component services of mashup m appearing in the top-i results of the recommended ranking list.

6.3 Results and Discussion

6.3.1 Impact of g

According to the statistics shown in Fig. 5, the number of services in most real-world mashups is between 1 and 4.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:59:02 UTC from IEEE Xplore. Restrictions apply.



(a) precision@N($N \leq 5$) (b) precision@N($N \leq 50$)

(c) recall@N($N \leq 5$) (d) recall@N($N \leq 50$)

(e) f1@N($N \leq 5$) (f) f1@N($N \leq 50$)

(g) MAP@N($N \leq 5$) (h) MAP@N($N \leq 50$)

Fig. 7. Comparative results between the seven methods and CSBR.

Therefore, the service number limitation of the service package (package size) is set to $g \in [1, 4]$ in our experiment.

To illustrate the impacts of different package size limitations in service bundle recommendation, we compare the performance of the proposed CSBR under different package size limitations. Fig. 6 shows that the performance and the number of service bundles (N_b) ranges from 1 to 10.

As g represents the maximum number of services included in a service package, when $g = 1$, there is only one service in a service package, i.e., the compositional semantics is not really captured. In Fig. 6a, $g = 1$ has a higher precision when recommending 1 or 2 bundles because the average number of services recommended at package size 1 is smaller than the other package size $\{2, 3, 4\}$. In Fig. 6b, regardless of the number of bundles, the recall of $g = 1$ is far smaller than the other sizes $\{2, 3, 4\}$. When $g = 2$, the proposed CSBR approach achieves the best performance. This indicates that the extraction of the compositional semantics of service packages makes sense and $g = 3$ achieves the best performance in $f1$ as shown in Fig. 6c.

6.3.2 Overall Performance Comparison

We compare the performance of the proposed CSBR approach against the seven aforementioned methods. As baseline algorithms generate recommendations according to predicted ranking information, in order to compare our method with the baseline algorithms in a fair way, we also provide our results in terms of ranking recommendations based on the semantic service package repository ($g = 3$).

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:59:02 UTC from IEEE Xplore. Restrictions apply.

TABLE 4
The Average Number of Services Based on Different Package Size Limitations

g \ N_b	1	2	3	4	5	6	7	8	9	10
1	1.14	2.27	3.43	4.58	5.77	6.86	8.01	9.20	10.33	11.45
2	1.56	3.13	4.43	5.80	7.06	8.35	9.60	10.83	11.98	13.13
3	1.71	3.22	**4.60**	6.01	7.38	8.61	9.91	11.21	12.47	**13.70**
4	1.66	3.17	4.54	5.89	7.30	8.66	9.95	11.29	12.58	13.91

Fig. 7 shows the performance of different algorithms with different recommended number N.

As shown in Fig. 7a, when N≤ 5, the precision of the proposed CSBR approach far exceeds the other seven methods. Regarding recall, CSBR also outperforms the other algorithms, which is shown in Fig. 7c. The performance of the other seven methods increases with an increase in the recommendation number. In particular, CDSR has the fastest increasing speed of all the comparative methods. When N$= 50$, its precision and recall are close to those of the proposed CSBR approach, as shown in Figs. 7b and 7d. Nevertheless, the competitive superiority of the proposed CSBR approach continues for different recommendation numbers, which indicates that the semantic service package repository is very useful for service recommendation in mashup development. Figs. 7e, 7f, 7g, and 7h present the f1 and MAP measure of different methods, which further validates the superiority of the proposed CSBR approach.

Table 4 presents the average number of services recommended under different package sizes g and the number of bundles N_b ranges from 1 to 10. When $g = 3$ and $N_b = 3$, an average of 4.6 services are recommended to the user. As shown in Fig. 6, precision is 0.153 and recall is 0.413. As shown in Fig. 7, the nearest recommendation number is 5, the highest precision is 0.131 (CDSR) and the highest recall is 0.37 (MRN). When $g = 3$ and $N_b = 10$, an average of 13.7 services are recommended to the user. As shown in Fig. 6, precision is 0.0647 and recall is 0.513. We compare the result with the recommendation number 20 in Fig. 7, showing the highest precision is 0.059 (CDSR) and the highest recall is 0.561 (CDSR). Even in this case, the gap in recall between the proposed CSBR approach and the optimal method is razor-thin at only 4 percent. This indicates that, by constructing the semantic service package repository, the proposed CSBR approach is more suitable for recommending appropriate services to meet the user's requirements.

6.3.3 Discussions

In the experiment, CSBR achieves much better performance than counterparts. The reason is those approaches can only learn superficial semantics of each service and fail to capture the compositional semantics of service packages. Unfortunately, the compostional semantics can not be inferred from descriptions of each service. In contrast, CSBR learns compositional semantics explicitly so that it can match the requirements of mashup with complimentary services.

CSBR relies on a set of techniques such as LDA, Gurobi toolkit, Superficial semantic similar relationship calculation and k-means. Essentially, CSBR is a framework, which means these techniques can be replaced with other ones with similar function. Although the implementations of CSBR introduced in this paper achieves much better performances than comparative ones, to make use of new techniques to implement CSBR is definitely meaningful.

Although CSBR can recommend a bundle of useful services, the compatibility among the recommended services is not considered explicitly. Fortunately, we obtain the service packages from existing service compositions, which is a way to guarantee the compatibility of these selected services.

Another limitation of CSBR is it only recommends services that have been used in existing mashups. However, there are more services that have not been used, especially when the number of new services developed every year increases quickly. A possible way is to provide some chances to these unpopular services. When a popular service is recommended as a component in a service bundle, unpopular services that have very similar functions can be recommended as a substitution with a certain probability. This issue will be explored in our future work.

7 Conclusion

In this paper, we study the problem of service bundle recommendation for mashup development. We present a compositional semantics-based model to recommend a set of complementary services for mashup creation. Due to the fact that the combination of different services generates compositional semantics, we focus on the construction of the semantic service package repository. Based on the constructed semantic service package repository, a bundle of complementary services is recommended to fulfill the functional requirements. We also experimentally confirm the effectiveness of our algorithm compared to the state-of-the-art methods on a real-world dataset. Interesting directions for future work include the prediction of the number of compositional semantics and cold-start service recommendation.

Acknowledgments

This work was supported in part by National Key Research and Development Plan under Grant 2018YFB1003802, and in part by China National Science Foundation under Grant 62072301.

References

[1] W. Xu, J. Cao, L. Hu, J. Wang, and M. Li, "A social-aware service recommendation approach for mashup creation," in *Proc. IEEE Int. Conf. Web Services*, 2013, pp. 107–114.
[2] N. Zang, M. B. Rosson, and V. Nasser, "Mashups: Who? what? why?," in *Proc. Extended Abstr. Hum. Factors Comput. Syst.*, 2008, pp. 3171–3176.
[3] L. Grammel and M. A. Storey, *A Survey of Mashup Development Environments*. Berlin, Germany: Springer, 2010.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:59:02 UTC from IEEE Xplore. Restrictions apply.

[4] S. Aghaee and C. Pautasso, "End-user development of mashups with naturalmash," *J. Vis. Lang. Comput.*, vol. 25, no. 4, pp. 414–432, 2014.

[5] C. Li, R. Zhang, J. Huai, and H. Sun, "A novel approach for API recommendation in mashup development," in *Proc. IEEE Int. Conf. Web Services*, 2014, pp. 289–296.

[6] A. Jain, X. Liu, and Q. Yu, "Aggregating functionality, use history, and popularity of APIs to recommend mashup creation," in *Proc. Int. Conf. Service-Oriented Comput.*, 2015, pp. 188–202.

[7] Y. Zhong, Y. Fan, W. Tan, and J. Zhang, "Web service recommendation with reconstructed profile from mashup descriptions," *IEEE Trans. Autom. Sci. Eng.*, vol. 15, no. 2, pp. 468–478, Apr. 2018.

[8] W. Gao, L. Chen, J. Wu, and H. Gao, "Manifold-learning based API recommendation for mashup creation," in *Proc. IEEE Int. Conf. Web Services*, 2015, pp. 432–439.

[9] P. Samanta and X. Liu, "Recommending services for new mashups through service factors and top-k neighbors," in *Proc. IEEE Int. Conf. Web Services*, 2017, pp. 381–388.

[10] B. Bai, Y. Fan, K. Huang, W. Tan, B. Xia, and S. Chen, "Service recommendation for mashup creation based on time-aware collaborative domain regression," in *Proc. IEEE Int. Conf. Web Services*, 2015, pp. 209–216.

[11] S. Abiteboul, O. Greenshpan, T. Milo, and N. Polyzotis, "MatchUp: Autocompletion for mashups," in *Proc. IEEE 25th Int. Conf. Data Eng.*, 2009, pp. 1479–1482.

[12] O. Greenshpan, T. Milo, and N. Polyzotis, "Autocompletion for mashups," *Proc. VLDB Endowment*, vol. 2, no. 1, pp. 538–549, 2009.

[13] B. Xia, Y. Fan, W. Tan, K. Huang, J. Zhang, and C. Wu, "Category-aware API clustering and distributed recommendation for automatic mashup creation," *IEEE Trans. Services Comput.*, vol. 8, no. 5, pp. 674–687, Sep./Oct. 2015.

[14] W. Gao and J. Wu, "A novel framework for service set recommendation in mashup creation," in *Proc. IEEE Int. Conf. Web Services*, 2017, pp. 65–72.

[15] A. Maaradji, H. Hacid, R. Skraba, and A. Vakali, "Social web mashups full completion via frequent sequence mining," in *Proc. IEEE World Congress Services*, 2011, pp. 9–16.

[16] S. R. Chowdhury, F. Daniel, and F. Casati, "Efficient, interactive recommendation of mashup composition knowledge," in *Proc. Int. Conf. Service-Oriented Comput.*, 2011, pp. 374–388.

[17] Q. Gu, J. Cao, and Q. Peng, "Service package recommendation for mashup creation via mashup textual description mining," in *Proc. IEEE Int. Conf. Web Services*, 2016, pp. 452–459.

[18] J. Cao, Y. Lu, and N. Zhu, "Service package recommendation for mashup development based on a multi-level relational network," in *Proc. Int. Conf. Service-Oriented Comput.*, 2016, pp. 666–674.

[19] L. Caruccio, V. Deufemia, S. Esposito, and G. Polese, "Combining collaborative filtering and semantic-based techniques to recommend components for mashup design," in *Comput. Intell. Semantic Knowl. Manage.*, 2020, pp. 25–37.

[20] P. Rodriguez-Mier , C. Pedrinaci, M. Lama, and M. Mucientes, "An integrated Semantic Web service discovery and composition framework," *IEEE Trans. Services Comput.*, vol. 9, no. 4, pp. 537–550, Jul./Aug. 2016.

[21] P. Rodriguezmier, M. Mucientes, and M. Lama, "Hybrid optimization algorithm for large-scale QoS-aware service composition," *IEEE Trans. Services Comput.*, vol. 10, no. 4, pp. 547–559, Jul./Aug. 2017.

[22] D. Roman, J. Kopeck, T. Vitvar, J. Domingue, and D. Fensel, "WSMO-Lite and hRESTS: Lightweight semantic annotations for Web services and RESTful APIs," *Web Semantics Sci. Services Agents World Wide Web*, vol. 31, pp. 39–58, 2015.

[23] A. V. Paliwal, B. Shafiq, J. Vaidya, H. Xiong, and N. Adam, "Semantics-based automated service discovery," *IEEE Trans. Services Comput.*, vol. 5, no. 2, pp. 260–275, Second Quarter 2012.

[24] B. Bai, Y. Fan, W. Tan, and J. Zhang, "DLTSR: A deep learning framework for recommendation of long-tail web services," *IEEE Trans. Services Comput.*, vol. 13, no. 1, pp. 73–85, Jan./Feb. 2020.

[25] X. Wu, Y. Fan, J. Zhang, H. Lin, and J. Zhang, "QF-RNN: QI-matrix factorization based RNN for time-aware service recommendation," *Proc. IEEE Int. Conf. Services Comput.*, 2019, pp. 202–209.

[26] S. Hu, Z. Tu, Z. Wang, and X. Xu, "A PoI-sensitive knowledge graph based service recommendation method," *Proc. IEEE Int. Conf. Services Comput.*, 2019, pp. 197–201.

[27] Y. Cao, J. Liu, M. Shi, B. Cao, T. Chen, and Y. Wen, "Service recommendation based on attentional factorization machine," *Proc. IEEE Int. Conf. Services Comput.*, 2019, pp. 189–196.

[28] C. Guo, Y. Xu, X. Hou, N. Dong, J. Xu, and Q. Ye, "Deep attentive factorization machine for app recommendation service," *Proc. IEEE Int. Conf. Web Services*, 2019, pp. 134–138.

[29] M. Shi, Y. Tang, Y. Huang, and M. Hua Lin , "Mashup tag completion with attention-based topic model," *Service Oriented Comput. Appl.*, vol. 15, pp. 43–54, 2021.

[30] M. R. U. Rahman and X. Liu, "Integrated topic modeling and user interaction enhanced WebAPI recommendation using regularized matrix factorization for mashup application development," *Proc. IEEE Int. Conf. Services Comput.*, 2020, pp. 124–131.

[31] B. Cao, M. Shi, X. Liu, J. Liu, and M. Tang, *Using Relational Topic Model and Factorization Machines to Recommend Web APIs for Mashup Creation*. Berlin, Germany: Springer, 2016.

[32] B. Cao, X. Liu, M. M. Rahman, B. Li, J. Liu, and M. Tang, "Integrated content and network-based service clustering and web APIs recommendation for mashup development," *IEEE Trans. Services Comput.*, vol. 13, no. 1, pp. 99–113, Jan./Feb. 2020.

[33] Z. Gao *et al.*, "SeCo-LDA: Mining service co-occurrence topics for recommendation," in *Proc. IEEE Int. Conf. Web Services*, 2016, pp. 25–32.

[34] L. Yao, X. Wang, Q. Z. Sheng, W. Ruan, and W. Zhang, "Service recommendation for mashup composition with implicit correlation regularization," in *Proc. IEEE Int. Conf. Web Services*, 2015, pp. 217–224.

[35] L. Yao, X. Wang, Q. Z. Sheng, B. Benatallah, and C. Huang, "Mashup recommendation by regularizing matrix factorization with API co-invocations," *IEEE Trans. Services Comput.*, vol. 14, no. 2, pp. 502–515, Mar./Apr. 2018.

[36] W. C. Mann and S. A. Thompson, "Rhetorical structure theory: Toward a functional theory of text organization," *Text Talk*, vol. 8, no. 3, pp. 243–281, 1988.

[37] S. Joty, G. Carenini, and R. T. Ng, *Codra: A Novel Discriminative Framework for Rhetorical Analysis*. Cambridge, MA, USA: MIT Press, 2015.

[38] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent Dirichlet allocation," *J. Mach. Learn. Res. Arch.*, vol. 3, pp. 993–1022, 2003.

[39] C. H. Papadimitriou, P. Raghavan, H. Tamaki, and S. Vempala, "Latent semantic indexing: A probabilistic analysis," *J. Comput. Syst. Sci.*, vol. 61, no. 2, pp. 217–235, 2000.

[40] J. Cao, Y. Zhang, J. Li, and S. Tang, "A method of adaptive selecting best LDA model based on density," *Chin. J. Comput.*, vol. 31, no. 10, pp. 1780–1787, 2008.

[41] A. Agrawal, W. Fu, and T. Menzies, "What is wrong with topic modeling? and how to fix it using search-based software engineering," *Inf. Softw. Technol.*, vol. 98, pp. 74–88, 2018.

[42] M. V. Mantyla, M. Claes, and U. Farooq, "Measuring LDA topic stability from clusters of replicated runs," in *Proc. 12th ACM/IEEE Int. Symp. Empir. Softw. Eng. Meas.*, 2018, pp. 1–4.

[43] Optimization G. Inc., "Gurobi optimizer reference manual," 2015. [Online]. Available: http://www.gurobi.com

[44] T. M. Kodinariya and P. R. Makwana, "Review on determining number of cluster in k-means clustering," *Int. J. Adv. Res. Comput. Sci. Manage. Stud.*, vol. 1, no. 6, pp. 90–95, 2013.

[45] P. M. Pardalos and G. P. Rodgers, "Computational aspects of a branch and bound algorithm for quadratic zero-one programmingrechnerische aspekte eines branch and bound-algorithmus zur quadratischen null-eins-programmierung," *Computing*, vol. 45, no. 2, pp. 131–144, 1990.

[46] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the State-of-the-Art and possible extensions," *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 6, pp. 734–749, Jun. 2005.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:59:02 UTC from IEEE Xplore. Restrictions apply.

Qi Gu received the PhD degree from Shanghai Jiao Tong University, China, in 2020. She is currently a lecture of the School of Information Science and Technology, Nantong University, China. Her research interests include intelligence recommender system, service computing and software engineering.

Jian Cao (Senior Member, IEEE) is currently a tenured professor with the Department of Computer Science and Engineering, Shanghai Jiao Tong University. His research interests include intelligent data analytics, service computing, collaborative computing and software engineering. Besides national and provincial government research grants, his research is also supported by many industry partners. He has published more than 200 research papers in prestigious journals and conferences. Currently, he is the distinguished member of CCF.

Yancen Liu (Student Member, IEEE) is currently working toward the postgraduate degree at Shanghai Jiao Tong University, China. Her research interests include intelligence recommender system, natural language processing and software engineering.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:59:02 UTC from IEEE Xplore. Restrictions apply.